Technical Report:

1. Development of mathematical model

An integer linear programming model was developed to formulate the problem, starting from the Maximal Covering Location problem. Below the parameters and variables used to solve the problem are presented:

$d \in D$	days
$o \in O$	observation segments
$p \in P$	PIs
$q \in Q$	qualities (categorical)
N	max number of observation per day
qp	initial quality of PI (categorical)
wp	score of PI
V	set of coefficients for quality q
$vq \in V$	value of quality q
d(o)	day of observation o
q(o, p)	quality of point p in observation o
aop	1 if PI p belongs to observation o; 0 otherwise

The parameter N depends on the length of the observations, since we have to respect the constraint of maximum T seconds of data collected per day by Marsis observation. Hence, given L the length of an observation N = T/L
The set V is used to give the right weight to different qualities. Indeed, there is not a linear relation between quality value and its preference. The binary score of a PI represents how much the preference for this AOI is higher with respect to all the other general areas on the South Pole.

Two sets of binary variables are used in the model. The number of variables strongly influences the speed of the model and its complexity:

x_o = 1 if observation o is selected, 0 otherwise;
y_{pq} = 1 if point p is observed at least once with quality q, 0 otherwise;

The objective function of the problem tries to maximize the quality collected by the observation of the PIs. In the objective function the ypq binary variable is weighted by two factors. The first one is the quality gained with the coverage of that PI, computed as the difference between the

predicted quality of the observation that covered it and the initial quality, derived from past observations. The second factor is the importance of the PI. Both the weights, that belong to V and W respectively, can assume different values to evaluate different priority scenarios or can be removed to evaluate the simpler problem of coverage maximization.

$$\max \sum_{p \in P} \sum_{q \in Q: q > \bar{q}_p} y_{pq} \cdot (v_q - v_{\bar{q}_p}) \cdot w_p$$

$$s.t. \quad y_{pq} \leq \sum_{\substack{o \in O \\ q(o,p)=q}} x_o \cdot a_{op} \qquad p \in P, q \in Q \quad (1)$$

$$\sum_{q \in Q} y_{pq} \leq 1 \qquad p \in P \quad (2)$$

$$\sum_{\substack{o \in O \\ d_o = d}} x_o \leq N \qquad d \in D \quad (3)$$

$$y_{pq} \in \{0,1\} \qquad p \in P, q \in Q \quad (4)$$

$$x_o \in \{0,1\} \qquad o \in O \quad (5)$$

Constraints (1) relate the two binary variables, so that a PI can be considered covered with quality q only if it belongs to at least an observation which is selected and is associated with the same quality q. Constraints (2) prevent the model from collecting a quality gain for a PI more than once. In particular, we expect that the model will set to 1 the variable ypq when q is the highest possible for p. Constraints (3) express the main limitation of the mission, that is the maximum time of observation per day, here translated as the maximum number of observations that we can select in each day d. The last two constraints, (4) and (5), express the binary nature of the variables.

2. Development of heuristic algorithms

2.1 Heuristic algorithms

We proposed three constructive heuristics, that build the solution by evaluating several opportunities at each step and then iteratively update the solution. In particular, all these heuristics exploit a greedy procedure, i.e. at each step they select the observation that can add the maximum profit to the objective function. The first heuristic builds the solution by considering the days in order and filling them with the maximum number of observations possible. The second heuristic exploits the concept of fairness in the allocation of tasks to each day. Indeed, it analyzes all the days in order and assigns them one observation at a time: in N iterations it reaches to complete the schedule of each day. In the third heuristic we do not evaluate the observations in a daily time window. At each iteration the algorithm searches for the observation to select among all the observations, regardless of the day to which it belongs. After it finds the candidate observation, that is the one with higher profit, it has to check its

feasibility. An observation is feasible if it is possible to assign it to the corresponding daily schedule without violating the constraint of maximum observation time per day. If it is not possible, no observation is selected for that iteration and the process continues until the schedule is not completed.

2.2 Matheuristics

We developed matheuristic algorithms following the fix-and-optimize framework.The algorithms start with a feasible solution generated from one of the heuristic and iteratively attempt to improve it. In particular, in each iteration a simplified mathematical model is solved and provides the solution to a sub-problem. The simplification is obtained by fixing in the master model a certain percentage of variables to their previous values, i.e. the value they had in the solution generated by the previous iteration. In this way, we are able to limit the number of variables in the model, and allow it to find the optimum solution of the sub-problem in a short time. We created nine different variants of the matheuristics in which we varied how we select the variables to be fixed. Specifically, we implemented a random-based selection, a chronological-based selection, and a geographical-based selection. Each of the three is further detailed in three variants.

More details on the heuristic algorithms will be provided in the article that will be published.

3. Testing

We performed several computational tests to verify the efficiency of the proposed algorithms, analyze their performances and identify their limits. The algorithms were coded in Python and run on a server with Intel Xeon CPU E5-2640 v3 3.60 GHz and 64 GB RAM under Windows 10 pro. The optimization was executed by Gurobi 950 Optimizer. For the model we set a time limit of 3600 seconds and we solved it with a barrier algorithm. In the local search, each iteration had a time limit of 360 seconds and the total execution time was limited to 3600 seconds too.

First of all, we compared the three heuristic algorithms. We notice that heuristic H1 and H2 always give the same results when considering a single observation per day. For this reason we discarded H2 as it was slower.
We then tested the performance of the nine matheuristic of the biggest instance in terms of objective value and found the most promising variant, called R2. This variant was used for the comparison with the other optimization algorithms.
Finally, we compared the performance of the model, the two heuristics, and the best matheuristic on the 4 generated instances (3 small and 1 big). For the big instance all the heuristics gave good results, in particular H3, that always finds a higher solution than H1. On the other hand, H1 is very fast while H3 can reach more than 1 hour of execution. The matheuristic is surely the best method. Indeed, it found the best in about 1 hour. The model showed all its limits with the bigger instance, for which it ended the execution with a huge gap from the upper bound. For the smaller instances, the results showed some differences. Indeed, the heuristics still remained

good methods, reaching good solutions or even the optimum in few seconds. Besides, the model and the matheuristics always found the optimal solution. In particular, we can say that the performances of the model are excellent, as it reaches the optimum in very low computational time, respect to other methods.

Detailed computational results will be available after the end of the test and the submission of the paper.